CMPSC 448 - Midterm Project Report

Identifying LLMs from Their Responses

Team Leader: Jacob Meert
Kasra Ghadimi
Jason McKinlay
Bryan Miller

1. Problem Definition

In recent years, Large Language Models (LLMs) have increasingly become part of everyday workflows for writing, brainstorming, and programming. Leading models like OpenAI's ChatGPT, Google's Gemini, and Anthropic's Claude come from different companies with their own training pipelines, safety fine-tuning, and system prompts. With this, each model ends up with subtle writing habits or stylistic quirks, which can show up across both surface-level formatting, like markdown lists, code blocks, and spacing, as well as general writing tone, such as vocabulary diversity, and cautious phrasing.

Figuring out whether generated text can be accurately traced back to the model that wrote it is a fundamental problem for text attribution, detecting AI-generated content, and understanding model behavior. The main goal of our project is to build and test neural network classifiers that can read an AI response and predict which model generated it.

We decided to approach this as a 3-class classification task. Using interactions pulled from the LMSYS Chatbot Arena preference dataset, each sample consists of a prompt and response pair, along with the true author label. Our classifiers take in the text tokens and predict class probabilities across the three models.

To break down how well models can be identified, we focused on four key research questions:

1. RQ1: Can a classifier accurately distinguish between outputs from various LLMs, relying solely on the generated text?
2. RQ2: Does the user prompt itself help to predict the target LLM?
3. RQ3: Do LLM fingerprints generalize across varying tasks?
4. RQ4: Which specific linguistic characteristics can we notice between LLMs?

2. Dataset Curation

The initial phase of our project included research towards a dataset to be used in order to train our models. We ultimately decided to approach this by selecting a pre-made dataset, rather than the alternative option of creating our own.

2.1 Source

For our dataset, we pulled from the LMSYS Chatbot Arena human preference dataset, arena-human-preference-140k, published by LMArena on Hugging Face in 2025 (https://huggingface.co/datasets/lmarena-ai/arena-human-preference-140k).

When deciding how to source our data, we initially considered querying the model APIs or manually prompting web interfaces ourselves to generate a custom dataset. However, doing so introduces a couple of big issues. Firstly, writing the prompts ourselves would reflect our own prompting habits, which could introduce bias and limit the diversity of tasks. Relying on API

access could also introduce API cost constraints and potential rate-limiting bottlenecks when scaling up to thousands of high-quality samples. Using the LMSYS Arena corpus instead gives us access to a massive, crowdsourced distribution of authentic, real-world user prompts covering multiple topics, such as coding, creative writing, reasoning, and open-ended Q&A, without artificial prompting patterns.

In its raw format, the dataset consists of pairwise battles where two anonymous models, model_a and model_b, generate responses to the exact same user prompt, and a human evaluator casts a vote on which completion is better. While LMSYS originally gathered these comparisons to evaluate model strength and calculate Elo ratings, the underlying data gives us real user queries side by side with real model outputs. This structure makes it straightforward to unpack the battles into the individual triplets we need to train and evaluate our classifiers, which are LLM name, input, and output.

2.2 Collection and Filtering

To turn the raw battle data into a clean set of training examples, we set up a filtering pipeline designed to remove noise and make sure our models learn genuine stylistic signals rather than superficial shortcuts. First, we isolated three distinct model families using their exact model version strings from the dataset:

- Claude: claude-opus-4-20250514
- Gemini: gemini-2.5-flash
- ChatGPT: chatgpt-4o-latest-20250326

We specifically chose these three checkpoints because they had the highest volume of inputs and outputs in the entire 140k dataset for their respective families, giving us the deepest and most reliable pool of natural prompt response pairs to sample from without having to blend different sub-versions together. Grouping multiple model versions under one label, like mixing smaller variants with more powerful, flagship models, dilutes the actual writing style of a model family and makes classification less reliable. From there, we applied three strict quality controls to clean the data:

1. Strict Single Turn Only: We checked full_conversation and conv_metadata to confirm that each interaction had zero prior context, keeping only samples with a length of two, which includes strictly one prompt and one reply. Multi-turn dialogues carry earlier context, conversational steering, and user feedback, which introduces conversational noise and distorts how the models normally write.
2. English Only: We filtered strictly for dialogue in English. If multilingual text is left in, classifiers often latch onto language differences across models rather than actual sentence structure and tone.

3. Length Bounding (50 to 800 Words): This range removes extreme lengths at both ends. The models still differ in typical length within it (Section 2.3), and length remains one of the signals our classifiers use (Sections 5.1 and 6.2).
   - Lower bound (≥ 50 words): Strips out trivial one-liners, short acknowledgments, and refusal messages that do not have enough text to show a real writing style.
   - Upper bound (≤ 800 words): Eliminates massive code dumps and oversized essays, which would be heavily truncated at our 384-token input length (Section 3.1) and are less representative of typical responses.

2.3 Balance and Splits

To keep the classes completely even and avoid any model getting favored during training, we balanced our dataset to exactly 700 examples per model family, giving us 2,100 total samples across Claude, Gemini, and ChatGPT. We formatted every entry into the required triplet structure: LLM_name, LLM_Input, and LLM_output.

From there, we split the 2,100 samples into train, validation, and test sets using a stratified 70/15/15 split with a fixed random seed (seed=42) so our partitions stayed balanced and repeatable across runs:

- Training Set (70%): 1,470 total rows, with exactly 490 examples per model.
- Validation Set (15%): 315 total rows, with exactly 105 examples per model.
- Test Set (15%): 315 total rows, with exactly 105 examples per model.

While our dataset is strictly balanced in terms of sample counts, the models still show distinct behavior in how much text they generate within our 50 to 800 word window. As illustrated in Figure 1 below, Claude tends to be noticeably more concise, with a tight distribution peaking around a median of 218 words. In contrast, ChatGPT and Gemini produce substantially more verbose answers, recording medians of 372 words and 417 words, respectively. Because ChatGPT and Gemini share significant distributional overlap across the 300 to 600 word range, our classifiers cannot rely solely on sequence length to distinguish between them, requiring the neural networks to learn deeper lexical and stylistic patterns instead.

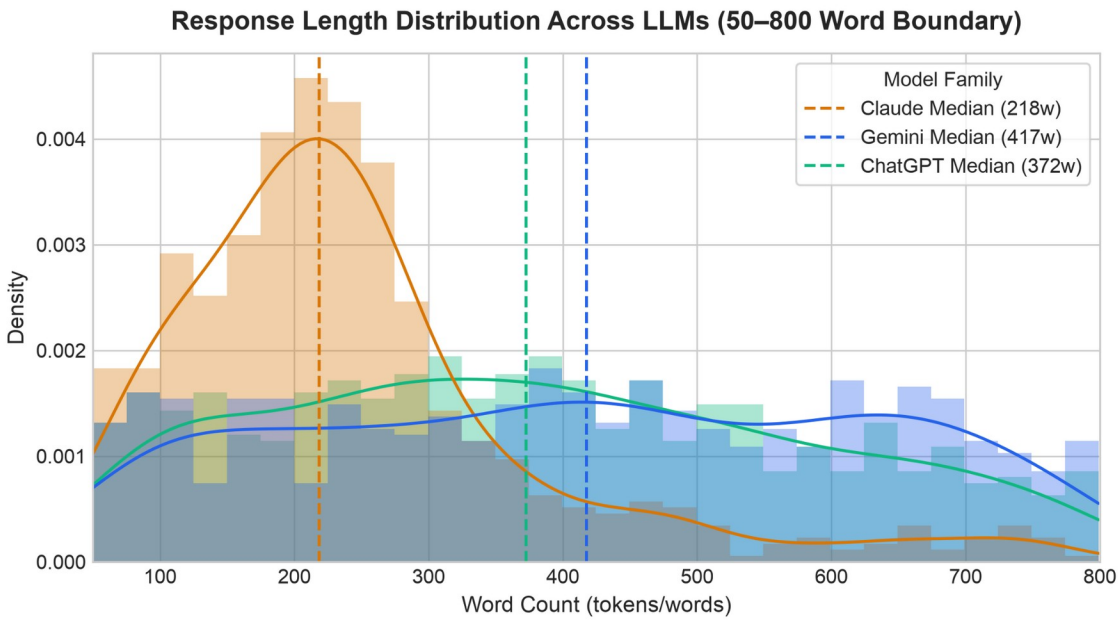


Figure 1. Response length distribution by LLM within the 50 to 800 word filter.

2.4 Data Notes

Because our data comes directly from pairwise Arena battles where two models responded to the exact same question, there is natural partial prompt overlap across the different classes. This overlap proves to be quite useful for our experiments, especially when we later delve into RQ2. It lets us test whether the models are learning how different LLMs uniquely handle identical prompts, or if seeing the user input creates an artificial shortcut that leaks the model label.

In terms of data integrity and safety, the original LMSYS dataset was already scrubbed under crowdsourced terms to strip out personally identifiable information. We additionally looked through batches of the inputs and outputs to make sure the dataset didn't include any private user records, sensitive medical information, confidential credentials, or copyrighted proprietary code that shouldn't be used for the sake of our project.

3. Model Implementation and Training

To evaluate model attribution across different neural architectures, we implemented and trained both a 1D CNN and an RNN using a shared text preprocessing pipeline.

3.1 Shared Preprocessing

Both models read text through the same three steps. First, each response is lowercased and split by a regular-expression tokenizer that makes every word one token and every punctuation mark or symbol its own token. We keep punctuation and markdown characters such as *, #, and backticks, and apply no stemming or stop-word removal, because formatting and function words may be part

of what distinguishes one LLM's writing from another's. Lowercasing keeps the vocabulary small for a dataset of this size.

Second, the vocabulary is built from the training split only, so no information from the validation or test sets reaches the model. Tokens that appear at least twice in the training data are ranked by frequency and kept up to a size cap, which removes one-off tokens the model could only memorize. Two reserved indices stand for padding and for tokens outside the vocabulary, and the cap includes them. The cap is the only preprocessing setting that differs between the two models: 20,000 tokens for the CNN and 250 for the RNN, as explained in Sections 3.2 and 3.3.

Third, every sequence is fixed at 384 tokens so that examples can be batched: longer responses keep their first 384 tokens and shorter ones are padded at the end. We chose 384 as a conventional length and did not tune it. At this length, 57.9% of responses are truncated, so the models see the full text of the rest and only the opening of these.

3.2 CNN

Our CNN is a 1D TextCNN built in PyTorch. CNNs are usually associated with images, but sliding 1D filters over a sequence of word embeddings lets the model pick up short phrases and formatting patterns wherever they appear in a response. It uses a 20,000-token vocabulary.

Each token ID is mapped to a 128-dimensional learned embedding. The sequence then goes through three parallel convolutional layers with kernel sizes of 3, 4, and 5 tokens and 100 filters each, so the model looks at 3-, 4-, and 5-token windows at the same time. Each layer is followed by a ReLU and global max-pooling over the sequence, which keeps the strongest response of each filter no matter where it occurred. The three pooled outputs are concatenated into a 300-dimensional vector, passed through dropout, with $p = 0.5$, and mapped by a linear layer to three logits, one per class. The model has 2,714,803 trainable parameters: 2,560,000 in the embedding, 153,900 in the convolutional layers, and 903 in the output layer.

We trained with cross-entropy loss and Adam for 15 epochs with a batch size of 32 on an Apple MPS GPU, using the 70/15/15 split from Section 2.3. After each epoch, we checked validation loss and saved the weights whenever it reached a new low. The test set was evaluated once, on that saved checkpoint, and played no part in model selection or tuning. Because training on MPS is not fully deterministic, we ran this configuration with three random seeds (42, 43 and 44). Test accuracy was $94.29 \pm 0.32\%$, ranging from 93.97% to 94.60%, and test loss was 0.1474 ± 0.0032.

3.3 RNN

Our RNN is a single-layer, unidirectional LSTM. We chose an LSTM over a plain RNN because its gates make it much less prone to vanishing gradients over sequences as long as ours. It uses a 250-token vocabulary, chosen through the tuning study in Section 5.3. The two models were tuned separately, which is why their vocabulary sizes differ.

Token IDs go through a 128-dimensional embedding, with the padding vector fixed at zero, and then an LSTM with a hidden size of 64 that reads left to right. Because padding sits at the end, we pack each batch by its true sequence lengths so the LSTM stops at the last real token instead of running through padding. The hidden state at that point acts as a summary of the response. It goes through dropout and a linear layer that outputs one logit per class. The model has 81,859 trainable parameters: 32,000 in the embedding, 49,664 in the LSTM, and 195 in the output layer.

Training followed the CNN setup with the same split, loss, learning rate, batch size, number of epochs, and checkpoint rule, with only two differences. We used AdamW with weight decay set to 0, which behaves the same as Adam, and we clipped gradients to a maximum norm of 1.0, because backpropagating through up to 384 time steps can make gradients explode. Seeds were fixed for Python, NumPy, and PyTorch, and training ran on an Apple MPS GPU. Across three seeds (42, 43 and 44), validation loss reached its minimum of 0.4506 ± 0.0159 between epochs 10 and 14, with 87.8% training accuracy and 83.4% validation accuracy at that point, and test accuracy was $85.82 \pm 0.73\%$.

3.4 Hyperparameter Summary

Table 1 lists the settings of both models side by side. The two models share their preprocessing, sequence length, dropout rate, learning rate, batch size, number of epochs and checkpoint rule. The main difference is vocabulary size: the RNN uses 250 tokens instead of 20,000, chosen by the tuning study in Section 5.3 to reduce overfitting. As a result, the CNN has about 33 times more parameters, almost all of them in its embedding table.

Table 1. CNN and RNN hyperparameters.

Hyperparameter	CNN (TextCNN)	RNN (LSTM)
Vocabulary size (incl. pad and unknown)	20,000	250
Sequence length (tokens)	384	384
Embedding dimension	128	128
Core layer	1D convolutions, filter widths 3, 4 and 5, 100 filters each, ReLU, global max-pooling	1-layer unidirectional LSTM, hidden size 64, packed sequences
Dropout	0.5 (before output layer)	0.5 (before output layer)
Optimizer	Adam	AdamW, weight decay 0 (equivalent to Adam)
Learning rate	1e-3	1e-3
Gradient clipping	None	Max norm 1.0
Batch size	32	32
Epochs	15	15
Loss	Cross-entropy	Cross-entropy
Checkpoint selection	Lowest validation loss	Lowest validation loss
Trainable parameters	2,714,803 (embedding 2,560,000; convolutions 153,900; output 903)	81,859 (embedding 32,000; LSTM 49,664; output 195)

4. Results

This section presents our findings across both neural network architectures, evaluating accuracy on isolated outputs and also determining whether incorporating prompt text impacts classification.

4.1 RQ1: Identifying the LLM from Its Output

Our first research question investigates whether neural network classifiers can accurately determine which LLM authored a text sample by looking solely at the model's generated response. In this setup, models are given no prompt context, metadata, or explicit author labels, meaning all predictions rely entirely on learned textual features, syntax, and formatting.

Because our test split contains exactly 105 samples for each of the three target classes, a baseline that randomly guesses yields an accuracy of exactly 33.33%. Both our 1D TextCNN and LSTM models substantially outperformed this baseline, proving that distinct, detectable LLM fingerprints exist in raw output text.

Table 2 below outlines the complete test performance metrics across both model architectures, evaluated on the held-out test set.

Table 2. Test performance on LLM output classification, mean ± std over seeds 42, 43 and 44.

Model	Test Loss	Test Accuracy	Macro Precision	Macro Recall	Macro F1-Score
1D TextCNN	0.1474 ± 0.0032	94.29 ± 0.32%	0.9431	0.9429	0.9429
LSTM (RNN)	0.4251 ± 0.0255	85.82 ± 0.73%	0.8588	0.8582	0.8582

The 1D TextCNN proved to be the stronger architecture overall, achieving an accuracy of 94.29% and a test loss of 0.1474, compared to 85.82% accuracy and 0.4251 loss for the LSTM.

Across multiple runs, the CNN consistently converged to a tighter loss and higher accuracy. The primary driver behind this performance difference comes down to how each architecture captures text patterns.

The CNN's parallel filters detect short phrases and formatting patterns anywhere in a response (Section 3.2), which suits the local, stylistic nature of LLM fingerprints.

The gap is not explained by vocabulary size. The LSTM uses a compact 250-token vocabulary chosen to prevent overfitting (Section 5.3), but when the CNN was restricted to the same 250 tokens it still reached 94.6% (Section 5.2), and giving the LSTM a larger vocabulary made it worse, not better. The difference therefore comes from the architectures. The LSTM must carry information through up to 384 sequential steps into a single final state, which likely loses some of the local cues that the CNN's max-pooling picks up directly.

To see whether specific model styles were easier or harder to detect, Table 3 breaks down class-level precision, recall, and F1-scores across each LLM family.

Table 3. Per-class classification performance on output text, mean over seeds 42, 43 and 44 (105 test responses per LLM per seed).

Model Architecture	Target LLM	Precision	Recall	F1-Score	Test Support
1D TextCNN	ChatGPT	0.9589	0.9619	0.9604	105
1D TextCNN	Claude	0.9222	0.9397	0.9308	105
1D TextCNN	Gemini	0.9481	0.9270	0.9374	105
LSTM (RNN)	ChatGPT	0.8828	0.8794	0.8808	105
LSTM (RNN)	Claude	0.8386	0.8730	0.8554	105
LSTM (RNN)	Gemini	0.8550	0.8222	0.8382	105

Both architectures found ChatGPT the easiest LLM to identify, with F1-scores of 0.9604 for the CNN and 0.8808 for the LSTM, while Claude and Gemini were harder to separate from each other (CNN 0.9308 and 0.9374; LSTM 0.8554 and 0.8382). ChatGPT uses the most newlines, code blocks and punctuation of the three models (Section 6.2), which gives both architectures a distinctive formatting signal, and the LSTM's small vocabulary, made up mostly of punctuation and markdown (Section 5.3), is well suited to picking it up.

The confusion matrices in Table 4 and Table 5 illustrate exact misclassifications across test samples.

Table 4. 1D TextCNN confusion matrix, summed over seeds 42, 43 and 44 (accuracy 94.29 ± 0.32%).

CNN	Predicted ChatGPT	Predicted Claude	Predicted Gemini	Total
Actual ChatGPT	303	7	5	315
Actual Claude	8	296	11	315
Actual Gemini	5	18	292	315

Table 5. LSTM (RNN) confusion matrix, summed over seeds 42, 43 and 44 (accuracy 85.82 ± 0.73%).

RNN	Predicted ChatGPT	Predicted Claude	Predicted Gemini	Total
Actual ChatGPT	277	17	21	315
Actual Claude	17	275	23	315
Actual Gemini	20	36	259	315

The confusion matrices, summed over our three seeds, show where the networks made errors. The CNN labeled 891 of 945 test predictions correctly, and more than half of its 54 errors were between Claude and Gemini (11 Claude responses predicted as Gemini and 18 Gemini responses predicted as Claude). ChatGPT was rarely confused with either, with only 12 of its 315 responses misclassified. The LSTM followed the same pattern on a larger scale: Claude and Gemini accounted for 59 of its 134 errors (23 and 36 in each direction), while ChatGPT stayed the most distinct, with 277 of 315 responses correct.

Our experiments for RQ1 lead to three clear conclusions. Firstly, neural networks can reliably attribute standalone AI responses back to their source model far above random chance without needing user prompts or conversational history. Secondly, parallel convolutional filters proved to be particularly well-suited for stylistic attribution, capturing layout cues and distinctive phrasing patterns more effectively than recurrence over long sequences. Finally, both architectures identify ChatGPT most easily and confuse Claude and Gemini most often.

4.2 RQ2: Does the Prompt Help?

We trained each model on three versions of the text: the prompt only, the response only, and the prompt followed by a separator token and the response. In the combined setting, the prompt is capped at 128 tokens, which is about its 90th-percentile length, so long prompts cannot push the response out of the 384-token window. Within each model, all settings use the same splits, hyperparameters, and checkpoint rule; only the input text changes. Each configuration was averaged across runs with seeds 42, 43, and 44.

Table 6. Test accuracy (%) by input setting, mean ± std over 3 seeds. Chance = 33.3%

Model	Input only	Output only	Input + Output
CNN	34.0 ± 4.4	94.3 ± 0.3	94.9 ± 1.1
RNN	35.1 ± 1.3	85.8 ± 0.7	83.1 ± 1.6

(i) Can the LLM be predicted from the prompt alone?

No. Both models are as accurate as chance (CNN 34.0%, RNN 35.1%), and their predictions are spread across all three classes. This is expected, as on LMArena the user writes the prompt before the two models are randomly assigned, so the prompt carries no information about which model answers.

(ii) Does including the input improve classification?

No. The CNN gains 0.6 points, which could be attributed to seed-to-seed variation, and the RNN drops by 2.7 points (83.1% vs. 85.8%), lower in every seed. Adding the prompt therefore does not help and can actually hurt. The real identifying signal lies in the LLM's response.

(iii) Why might input-only classification work surprisingly well?

It does not here, actually. In other datasets, input-only accuracy could be inflated if each model's prompts were collected differently from other time periods or user groups, for example, if prompts were labeled with the specific model it addresses. Additionally, we checked for leakage. Because each LMArena prompt is sent to two different models, 26 of our validation and test prompts also appear in the training set, but in 25 of them the training copy was answered by a different LLM. A model that memorized training prompts would therefore get these examples wrong, so this overlap could only have lowered input-only accuracy, not raised it.

5. In-Depth Analyses and Experiments

Beyond overall test benchmarks, we conducted focused error analyses and ablation studies to identify failures and isolate the primary drivers of model performance.

5.1 Error Analysis

Most of our models' mistakes happen on responses that do not look like their LLM's usual writing. Looking at every test prediction from one separate seed-42 run of each model (15 errors for the CNN and 46 for the RNN), we found that the two models often get the same responses wrong, and that the style-only classifier from Section 6.2 also misses most of them. The hard cases, in other words, are responses that break their LLM's normal habits.

Length shows this most clearly (Table 7). Claude usually writes short answers, so its long answers were misclassified much more often than its short ones. Gemini and ChatGPT usually write long answers, so their short ones were the problem, and were mostly mistaken for Claude. For example, a long, heavily bulleted Claude answer to "What is the human brain" was labeled ChatGPT, and a

short, plain Gemini refusal was labeled Claude by both models. This is a pattern we observed rather than one we tested directly.

Table 7. Error rate by true LLM and whether the response is longer than 384 tokens.

True LLM	CNN, over 384	CNN, under 384	RNN, over 384	RNN, under 384
Claude	9.4%	1.4%	31.2%	6.8%
Gemini	3.8%	14.8%	11.5%	25.9%
ChatGPT	2.6%	7.1%	11.7%	21.4%

5.2 CNN Ablation and Sensitivity Analysis

Testing changes to filter sizes, dropout, and vocabulary size showed that our CNN is stable across settings, keeping test accuracy between 94.6% and 94.9% across all runs. All runs in this study used the setup in Section 3.2, changed one setting at a time, and were evaluated on the test set using the checkpoint with the lowest validation loss. These ablations were run before our final three-seed evaluation, so the baseline here is a separate run (94.92%) from the three-seed results reported in Section 3.2.

Our baseline model combines three parallel filter sizes of 3, 4, and 5 words with 50% dropout and a 20,000-token vocabulary, reaching a test loss of 0.1559 and 94.92% accuracy. Testing single filter sizes showed that short 3-word windows alone can reach 94.92% accuracy, but the test loss rose to 0.2816. Moving to a 5-word filter brought the test loss back down to 0.1978, with a small change in accuracy to 94.60%. Stacking all three filter sizes gave the lowest test loss overall, showing that having both short and longer phrase windows helps the model make more confident predictions.

Table 8. CNN ablation study results on the test set across filter sizes, dropout regularization, and vocabulary limits.

Setup	Filter Output Size	Training Accuracy	Test Loss	Test Accuracy
Only 3 Word Filters (vocab 20k)	100	97.48%	0.2816	94.92%
Only 5 Word Filters (vocab 20k)	100	97.62%	0.1978	94.6%
No Dropout (vocab 20k)	300	100.00%	0.1713	94.6%
Small Vocab (250 words)	300	98.5%	0.1638	94.6%
Baseline (sizes 3, 4, 5, vocab 20k)	300	98.71%	0.1559	94.92%

Turning off dropout completely caused the model to memorise the training split, hitting 100.0% training accuracy with a training loss of 0.0030 by epoch 15. Without dropout, test loss rose from 0.1559 to 0.1713, and test accuracy dropped to 94.60%. Keeping dropout at 0.5 prevents the 300

pooled feature maps from co-adapting, protecting generalization across our 1,470 training examples.

The most notable result came from cutting the vocabulary from 20,000 down to 250 tokens to match the RNN in Section 5.3. Even with 98.8% of the vocabulary removed, the CNN achieved 94.60% test accuracy and a test loss of 0.1638. This shows that the performance gap between the two models does not come from vocabulary size. While the recurrent model struggled with long sequences unless restricted to common words, the CNN readily separates all three model families using only high-frequency words, punctuation, and local formatting patterns.

5.3 RNN Vocabulary Size and Overfitting

Shrinking the RNN's vocabulary from 20,000 to 250 tokens was the most effective change we tested, lowering mean best validation loss from 0.691 to 0.461 and raising validation accuracy from 74.0% to 83.3%. All runs in this study used the setup in Section 3.3 with a hidden size of 64, changed one setting at a time, and were repeated with seeds 42 and 43 (24 runs in total). The test set was not used.

With 20,000 tokens, 98.1% of the model's 2,609,859 parameters sit in the embedding table, and the model overfit early: validation loss bottomed out at epochs 7 to 8 and then rose, with an average gap of 13.2 points between training and validation accuracy. A smaller embedding (64), embedding dropout (0.3), and weight decay (0.1) did not reliably help. Weight decay gave the largest gain (0.653 vs. 0.691), but this was within the seed-to-seed variation of up to 0.069 we observed at this vocabulary size.

Reducing the vocabulary, by contrast, improved validation loss at every step down to 250 tokens before it worsened again at 100 and 50 (Figure 2), and it narrowed the accuracy gap to 3.6 points. At 20,000 tokens, the validation set had about twice the unknown-token rate of the training set (5.8% vs. 2.6%), meaning many vocabulary words appeared only in training and could be memorized; at 250 tokens the rates matched (38.0% vs. 37.9%). The 250 and 100-token settings (0.461 vs. 0.474) cannot be reliably separated, and three runs of the identical 250-token configuration with the same seed varied by 0.016 due to MPS nondeterminism; we chose 250 for its lowest mean and most consistent seeds. By token volume, 91% of the in-vocabulary tokens are punctuation, markdown symbols, or function words, although 82 of the 248 entries are content words, many of them programming terms. This suggests the RNN relies mainly on formatting and function-word habits, which Section 6.2 examines further.

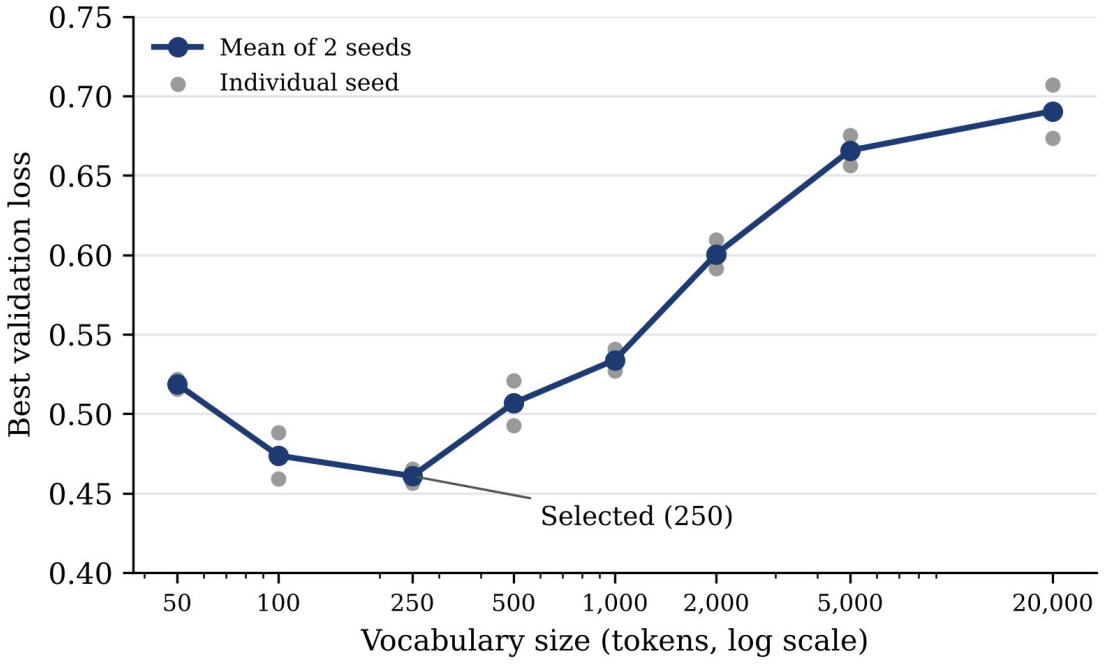


Figure 2. Best validation loss vs. vocabulary size for the RNN. Gray points are individual seeds; the line connects the two-seed means.

6. Extended Research Questions (Extra Credit)

Building upon our core evaluation, we investigated two additional research questions to test how our classifiers generalize to out-of-domain tasks and to quantify the specific linguistic characteristics that distinguish each LLM.

6.1 RQ3: Cross-Domain Generalization

RQ1 showed that our classifiers can identify the LLM, but not whether they learned habits of the LLM itself or patterns tied to the tasks in the training data. RQ3 separates the two by testing on tasks absent from training: a general fingerprint should keep working, while task-specific patterns should fall toward chance at a rate of 33.3%. Based on Section 5.3, we expected the RNN, whose input is mostly punctuation, markdown, and function words, to transfer better than the CNN, whose 20,000-token vocabulary contains many topic words.

We trained each model on code only and tested it on unseen code and on creative writing and math. The model, weights, and vocabulary are identical across these evaluations, so any gap measures the cost of the domain shift, and because the model sees no task variety, held-out accuracy cannot come from similar training tasks. Creative writing is far from code and math is close, which shows whether the cost grows with distance.

Domains come from the dataset’s annotations: code where is_code is true, and creative writing or math where category_tag["creative_writing_v0.1"]["creative_writing"] or

category_tag["math_v0.1"]["math"] is true and is_code is false, excluding responses in more than one domain. Applying the RQ1 filters to the full dataset gave 8,482 responses, which we balanced within each domain (448 code, 191 creative writing, and 160 math responses per LLM). Code was split 70/15/15 (314/67/67 per LLM); creative writing (573) and math (480) were test-only and played no part in training, checkpoint selection, or the vocabulary. Prompts shared between two LLMs were kept in training. Both models used the RQ1 hyperparameters, with seeds 42, 43 and 44.

Table 9 summarizes the results. Every held-out score is well above chance. Creative writing cost the CNN 14.1 points and the RNN 5.2, while math cost little or nothing, and the unknown-token rate (test tokens outside the code-built vocabulary) rose in the same pattern.

Table 9. Results for classifiers trained on code, tested on code (in-domain, n = 201) and on creative writing (n = 573) and math (n = 480), both held out. Accuracy is the mean ± sd over seeds 42, 43, and 44 (chance = 33.3%); drop is computed within each seed and then averaged; macro F1 is in percent. The last three columns give per-LLM recall from confusion matrices summed over the seeds.

Model	Test set	Accuracy (%)	Macro F1	Drop (pts)	Unk. tokens	Claude	Gemini	ChatGPT
CNN	Code	92.9 ± 1.7	92.9	n/a	6.6%	95%	92%	93%
CNN	Creative	78.8 ± 2.0	78.9	14.1	13.6%	82%	83%	71%
CNN	Math	89.7 ± 0.4	89.8	3.1	7.4%	90%	87%	92%
RNN	Code	69.3 ± 3.3	69.1	n/a	34.3%	64%	72%	72%
RNN	Creative	64.2 ± 2.1	63.2	5.2	45.7%	74%	77%	42%
RNN	Math	70.1 ± 1.6	70.0	-0.8	35.3%	69%	77%	65%

(i) Does performance decrease on unseen tasks?

Yes, by an amount that grows with distance from the training domain. A likely reason is formatting: math answers, like code answers, tend to use markdown, symbols and step-by-step structure, while creative writing is mostly continuous prose, so fewer of the formatting cues learned from code are present. The rise in the RNN's unknown-token rate on creative writing fits this, since most tokens it recognizes are punctuation and markdown, though we did not measure formatting per domain. The RNN's creative-writing drop is not robust, however: seed 42 scored higher on creative writing than on code.

(ii) Are some models more consistently identifiable across domains?

Yes, Claude and Gemini stayed identifiable in all three domains. ChatGPT was well identified on math but not on creative writing, where its recall fell from 93% to 71% for the CNN and from 72% to 42% for the RNN, mostly to Gemini, so the cues learned for ChatGPT from code answers appear tied to technical responses. This matches Section 6.2: ChatGPT uses the most newlines, code blocks, and punctuation, cues that prose largely lacks. The CNN's ChatGPT F1 hides this because its ChatGPT predictions were 94% precise.

(iii) Is the classifier learning a general LLM fingerprint or task-specific patterns?

Mostly a general fingerprint. Trained on code alone, the classifiers identified the LLM in 64% to 90% of responses from unseen tasks; the CNN's 14.1-point loss on creative writing is the task-specific part. The RNN lost less, as predicted, with two caveats: 136 of the 248 entries in its code-built vocabulary are content tokens such as "python" and "import", though by occurrence, 66% of its tokens are punctuation or markdown and 20% are function words. Its lower in-domain accuracy left it less to lose. As the models also differ in vocabulary size, this is consistent with a stylistic fingerprint but does not establish it.

For the experiment as a whole, RQ1 accuracy is best read as a ceiling, since part of it rests on topical content that does not transfer. Limitations: one training direction, three seeds, unverified domain labels, and a CNN still improving at its final epoch in two of three seeds.

6.2 RQ4: Distinguishing Characteristics

While the previous research questions established that our models can identify LLM signatures, they do not show which characteristics make the models distinguishable. Section 5.3 found that the RNN performs best with a small vocabulary made up mostly of punctuation, markdown and function words, which suggests that structure and style matter more than topic. To test this directly, we developed a linguistic style profiler to extract and quantify structural and stylistic markers across the test set.

For structural characteristics, we decided to analyze average word count, newline density, bullet point frequency, code block frequency, punctuation ratio, and lexical diversity measured via Type-Token Ratio (TTR). Linguistic characteristics are then analyzed by discourse markers normalized per 1,000 words, including hedges, boosters, transitions, personal pronouns in first-person, direct addresses such as "you" or "your", and grammatical modal auxiliary verbs. These observations yielded the following results:

Table 10. Structural and linguistic profile across test split responses (N=105 per model), with discourse markers being normalized per 1,000 words

Category	Feature	ChatGPT	Claude	Gemini
Structural	Average Word Count	331.05	241.9	390.51
Structural	Newlines	59.22	28.25	37.8
Structural	Bullets	11.73	5.72	10.03
Structural	Code Block %	23.81%	15.24%	15.24%
Structural	Punctuation Ratio	0.11	0.06	0.08
Structural	Type-Token Ratio (TTR)	0.55	0.6	0.52
Linguistic	Hedges (per 1k words)	2.15	3.24	4.17
Linguistic	Boosters (per 1k words)	1.72	2.09	2.56
Linguistic	Transitions (per 1k words)	0.75	1.01	1.79
Linguistic	First-Person (I/we per 1k)	9.75	6.19	7.53
Linguistic	Direct Address (you per 1k)	12.72	11.71	14.96
Linguistic	Modal Verbs (per 1k words)	10.53	13.15	10.01

(i) What signals does the classifier appear to use?

The extracted profiles showed clear model-specific behaviors. Gemini produces the longest average outputs at 390.51 words with heavy list formatting. Claude ends up being the most concise at 241.90 words, using about half as many formatting breaks like new lines or bullet points. ChatGPT features the highest newline density and code block frequency, favoring shorter, distinct technical segments.

When it comes to lexical diversity, Claude achieves the highest Type-Token Ratio of 0.60 with more variation in vocabulary and lower word repetition when compared to ChatGPT's ratio of 0.55 and Gemini's 0.52. Because TTR naturally decreases as responses get longer, part of Claude's higher score likely reflects its shorter outputs.

In terms of overall tone, Gemini exhibits a more academic tone, leading all models in both hedges and boosters. In contrast, ChatGPT hedges at roughly half of Gemini's rate, adopting a more direct tone. Gemini also uses more than double the transition density of ChatGPT, heavily relying on words like "furthermore", "additionally", or "consequently" to guide the reader through longer explanations.

ChatGPT exhibits the strongest conversational persona, using the most first-person pronouns at a rate of 9.75 per 1000 words. Claude minimizes self-reference, while Gemini leads in direct second-person addresses, mirroring an instructional tutoring style. Claude also leads in modals, or auxiliary verbs, at a rate of 13.15 per 1000, showing a preference for politer words like “could”, “would”, or “might”.

(ii) Can we classify responses when only given these characteristics?

To determine how effective each characteristic is, we also implemented a logistic regression classifier using only the 12 standardized numerical features from Table 10. This model receives no raw text, word embeddings, or sequential context.

Table 11. Logistic regression on the 12 style features, test set (overall accuracy: 68.57%).

Classifier	Precision	Recall	F1-Score	Test Support
ChatGPT	0.7128	0.6381	0.6734	105
Claude	0.6870	0.7524	0.7182	105
Gemini	0.6604	0.6667	0.6635	105
Macro Average	0.6867	0.6857	0.6850	315

Against an assumed baseline of 33.33% across the three classes, this classifier achieves a 68.57% test accuracy. This shows that structural habits and discourse marker distributions are informative on their own, reaching roughly double chance accuracy without any access to the raw text. It also provides a clear explanatory baseline for our deep learning models: while simple stylistic characteristics distinguish models at ~68% accuracy, our CNN and RNN use sequential n-gram filters and hidden recurrence to capture the remaining patterns, increasing overall classification performance.

(iii) Does removing certain signals substantially reduce classification performance?

When we test the importance of each feature using our linear baseline, removing structural features causes the largest performance drop. Training exclusively on the six linguistic discourse markers lowers test accuracy from 68.57% down to 44.76%. Conversely, training on structural layout alone retains 68.89% accuracy. This shows that visual layout and text length are the main clues for identifying the model, while specific writing habits add minor details.

7. Lessons and Experience

Throughout the project, our team navigated challenges that provided us with many valuable insights.

7.1 Technical Lessons

With a small dataset, overfitting came from how many words a model could memorize. Shrinking the RNN’s vocabulary helped far more than any other change we tried, as Section 5.3 shows, and

we would now look at vocabulary size first when a text model overfits. We also learned to repeat runs across seeds before drawing conclusions, since several differences we first saw, such as the CNN's 0.6-point RQ2 gain and the weight decay result in Section 5.3, turned out to be within seed-to-seed variation.

7.2 Data Collection Challenges

Using a public dataset saved time, but its structure shaped our results. The fairest comparison gives every model the same prompts, so any difference comes from the model alone. LMArena only did this for a small number of our prompts, because each battle pairs two random models. Next time we would collect data where every model answers every prompt, and keep all answers to a prompt in the same split.

7.3 Team Experience

Splitting the work by research question let us move quickly, but it meant each of us trained and evaluated models in our own way, with different software versions, random seeds, and runs. The lesson we took away is to have everyone work from the same environment defined in our requirements.txt, and to agree on which runs count as our official results, so that every section of the report quotes the same numbers.

Additionally, working separately meant each of us built up some knowledge that the others did not have, which made combining our work harder than expected. Each person's conclusions made sense within their own experiment, but some did not hold once compared with the rest. For example, we initially attributed the CNN's advantage to its larger vocabulary, but a later test showed the CNN performed just as well with the RNN's small vocabulary, so its advantage comes from its architecture instead. Catching conflicts like this required explaining our experiments to each other in detail, and we learned that writing a joint report means checking each conclusion against everyone else's results, not just our own.